

UN Comtrade Extraction Pipeline

Project Guide and Handover Document

Table of Contents

1. Introduction
 2. Background: understanding the data source
 3. Project history: from the full history load to the yearly run
 4. How the pipeline is organised
 5. The settings file: `un_comtrade_config_v2.py`
 6. The extraction logic: `general_extractor.py`
 7. The Airflow job: `cp_extraction_comtrade_dag.py`
 8. The database tables
 9. The two limits of the source, and how they are handled
 10. Running and monitoring the pipeline
 11. Troubleshooting
 12. Glossary
 13. Frequently asked questions
-

1. Introduction

1.1 Purpose of this document

This document explains the UN Comtrade extraction pipeline to someone who has just joined the project and has never seen the code before. It assumes no prior knowledge of the data source and no prior knowledge of the code base.

After reading it, a new team member should understand what the pipeline does, why it is built the way it is, what each of the three files is responsible for, where the data ends up, and what to

do
when something goes wrong.

The document deliberately avoids going line by line through the code. The goal is to explain the
ideas, so that the code becomes easy to read on its own.

1.2 What the pipeline does in one paragraph

The pipeline downloads international trade statistics from the United Nations Comtrade database and stores them in our data warehouse. These statistics describe, for a given year, which country reported exporting or importing which product, with which partner country, in what quantity and for what value. The pipeline runs inside Airflow, keeps a record of everything it has already downloaded, and is designed so that it can be stopped and restarted at any moment without losing work or duplicating data.

1.3 Who uses the output

The data lands in a single raw table. From there, the transformation layer cleans it, reshapes it and builds the models used for reporting and analysis. The pipeline described in this document is therefore the **first** step of the chain: everything downstream depends on it being complete and correct.

2. Background: understanding the data source

Before explaining the code, it helps to understand the five ideas that shape the way the data is organised. Every design decision in the pipeline follows from them.

2.1 What UN Comtrade is

UN Comtrade is the United Nations database of official international trade statistics. Countries report their own trade figures to the UN, the UN harmonises them, and the result is

published
through a public web service. It is the standard reference source for trade analysis
worldwide.

Two properties of the source matter a great deal for us:

- **Countries report late.** A country's figures for a given year usually appear between six and eighteen months after that year ends. Some countries are quick, some take years, and a few never report at all.
- **Published data can change.** When a country sends a correction, the UN republishes that country's figures. Data that was downloaded a year ago is therefore not guaranteed to still match the source.

2.2 The reporter

The **reporter** is the country that reported the figures. Each reporter has an official numeric code, for example 842 for the United States. The list of reporters is published by the UN and changes over time, because countries appear, merge, split or are renamed.

The pipeline never hard-codes this list. It reads it from the UN before every run.

2.3 The partner

The **partner** is the country on the other side of the transaction. If Germany reports exporting goods to Japan, Germany is the reporter and Japan is the partner.

Partners matter to us for a second reason as well: when a download is too large, the pipeline splits it by partner. This is explained in section 9.

2.4 The trade flow

The **flow** says in which direction the goods moved. The pipeline downloads four flows:

Flow	Meaning
Import	Goods coming into the reporting country
Export	Goods leaving the reporting country

Re-import	Goods that were exported and have come back
Re-export	Goods that were imported and are sent on again

2.5 The classification

A **classification** is a system for grouping products into categories. The same shipment of goods can be described under several systems at the same time, depending on which system the analysis needs. The pipeline downloads all of them:

Classification	What it is	Data available from
HS	Harmonised System, the modern standard for goods	1988
S1, S2, S3, S4	Successive revisions of SITC, the older standard	1962
B4, B5	BEC, which groups goods by economic use	1962
SS	An additional goods breakdown	1962
EB	The classification used for services rather than goods	2000

This table explains a detail that often surprises newcomers: **the different classifications start in different years**. HS data simply does not exist before 1988, and services data does not exist before 2000. The pipeline knows each starting year and never asks for data that cannot exist.

2.6 The period

The pipeline downloads **annual** data. One period is therefore one calendar year.

3. Project history: from the full history load to the yearly run

3.1 Phase 1: the history load (completed)

The first phase of the project was a one-off operation: download everything that has ever

been
published, for every classification, going back to the earliest available year. That means
back to
1988 for HS, back to 1962 for the older goods classifications, and back to 2000 for
services.

This was a very large job. It involved hundreds of thousands of individual requests to the UN
web
service and ran over several days. Most of the safety machinery in the code exists because
of this
phase: a job that runs for days will inevitably be interrupted at some point, and it must be
able
to continue rather than start over.

This phase is finished. The historical data is in the warehouse and does not need to be
downloaded again.

3.2 Phase 2: the yearly run (current)

Because the history is complete, the pipeline now has a much smaller job: once a year,
download
the **previous year**. Running it in 2027 downloads 2026; running it in 2028 downloads 2027.

The year is calculated automatically from the current date. **Nobody needs to edit the code
each
year.** This was a deliberate decision: a configuration value that must be updated by hand
every
January is a value that will eventually be forgotten.

3.3 Both modes still exist

The code still supports the full history load. A single switch in the settings file chooses
between the two modes. Yearly mode is the default and the normal state of the project.

The history mode is kept for two reasons: if the warehouse ever has to be rebuilt from
scratch,
and if a decision is taken to re-download a range of old years after a major revision at the
source.

4. How the pipeline is organised

The project is made of three files, each with a clearly separated responsibility.

File	Responsibility	How often it is touched
<code>un_comtrade_config_v2.py</code>	What to download and where to put it	Occasionally
<code>general_extractor.py</code>	How to download it safely	Almost never
<code>cp_extraction_comtrade_dag.py</code>	When and in which order to run it	Rarely

One point about the middle file is worth making straight away. `general_extractor.py` is the **shared extractor used by several of our data sources**, not a file that belongs to Comtrade alone. It holds one function per source, and the Comtrade one is called

```
run_un_comtrade_api()
```

When this document talks about "the extraction logic", it means that function and the helper functions that belong to it, not the whole file.

The separation matters. Someone who needs to change the behaviour of the pipeline should, in

almost every case, be able to do it in the settings file without reading the logic. The extraction

logic contains the difficult parts (rate limits, splitting oversized downloads, resuming after a failure) and is intended to be left alone. It is also shared with other sources, so a careless change there can affect pipelines beyond this one.

The rest of this document explains each of the three files in turn.

5. The settings file: `un_comtrade_config_v2.py`

5.1 The idea

This file is a single dictionary of settings. Everything the pipeline needs to know that could reasonably change over time lives here: what to download, from which years, into which tables, and

under which limits. The extraction logic reads these values and never contains such decisions itself.

The practical benefit is that the pipeline can be reconfigured without any risk of breaking the

download logic.

5.2 What to download

The settings describe the scope of the extraction:

- **The type of trade:** goods and services. Both are downloaded.
- **The classifications,** listed per type: the goods classifications (HS, S1 to S4, SS, B4, B5) and the services classification (EB). Removing an entry from this list is all it takes to stop downloading that classification.
- **The trade flows:** imports, exports, re-imports and re-exports.
- **The frequency:** annual.

5.3 The starting year of each classification

As explained in section 2.5, each classification begins in a different year. The settings hold this mapping: HS from 1988, the older goods classifications from 1962, services from 2000.

New joiners often ask why these old years are still in the settings when the pipeline only downloads the previous year. There are three reasons, and none of them cause old data to be

downloaded again:

1. **They act as a guard.** The pipeline only requests a year for a classification if that year is on or after the classification's starting year, so it never asks for data that cannot exist.
2. **They describe the storage layout.** The data table is organised internally by year, and the starting year tells the pipeline which internal sections exist.
3. **They are needed if the history is ever rebuilt.** If the mode switch is ever set back to history, the correct starting years are already there.

5.4 Which mode to run

A single setting chooses between the yearly run and the full history load. It is set to the yearly

run, which is the normal state of the project.

5.5 The limits of the source

Two values record the rules imposed by the UN web service: the maximum number of requests per

second, and the maximum number of rows a single request may return. The extraction logic reads

these values rather than assuming them, so if the UN ever changes its rules, the pipeline can be adapted here. Section 9 explains how these limits shape the download.

5.6 Where things are stored

The settings name every table the pipeline writes to: the trade data table, the two reference tables holding the country lists, and the progress record. They also record the web addresses of the official UN reference files that the country lists are read from, so that if the UN ever moves those files, only this file needs updating.

5.7 Optional re-download

One setting allows a specific part of the data to be marked for re-download, for example a chosen classification, a chosen year or a chosen country. It is normally empty. It exists for the situation where someone discovers that a particular slice of data needs to be fetched again.

6. The extraction logic: `general_extractor.py`

6.1 Where the Comtrade logic lives

`general_extractor.py` is our shared extractor. It contains the extraction functions for several data sources, one function per source, so most of the file has nothing to do with Comtrade.

The Comtrade work lives in a single function:

```
run_un_comtrade_api()
```

It takes the classification to download and the range of years, and the Airflow job calls it once per classification. Everything described in this section happens inside that function and the helper functions that belong to it.

Two smaller functions in the same file support it and are described in section 6.9. They are used only by the history mode.

6.2 The idea

The guiding principle of the Comtrade logic can be stated in one sentence:

Download everything, never download the same thing twice, and never silently miss anything.

Every part of the function serves one of those three goals. The rest of this section walks through what happens when it runs.

6.3 Step one: refresh the country lists

Before a single piece of trade data is requested, the pipeline reads the official lists of reporting countries and partner countries from the UN and updates our copies in the database.

This step is deliberately first, because the list of countries determines everything that follows:

the pipeline asks for data country by country, so working from an outdated list would mean silently missing a country.

The update is careful about change:

- A country that appears for the first time is **added**.
- A country whose details have changed is **updated**.
- A country that has disappeared from the official list is **kept but marked inactive**, so that the historical data already downloaded for it still makes sense.
- If the UN adds a completely new field to its country list, the new field is **added automatically**, rather than being discarded.

The result is that the country list maintained in our database always reflects the source, without anyone having to watch the UN website for changes.

6.4 Step two: prepare the storage

The pipeline then makes sure the destination table exists, along with its internal structure.

The trade data table is very large, and a single unstructured table of that size becomes slow and

awkward to maintain. The table is therefore divided internally, first by classification and then by

year. Each combination, for example "HS data for 2026", is a separate internal section.

This brings three practical advantages:

- A query that asks for one classification and one year only reads that section, instead of scanning everything.
- Loading new data touches only one section, so it stays fast as the table grows.
- A single year can be removed and reloaded without disturbing the rest.

The new year's section is created automatically each time the pipeline runs, so no manual preparation is needed when a new year starts.

6.5 Step three: find out what is already done

The pipeline reads the **progress record**, a table that contains one row for every request it has ever made. It builds the list of everything that has already been downloaded successfully and skips all of it.

This is the single most important mechanism in the project. It is the reason a run can be interrupted at any moment and simply be started again.

In yearly mode there is one refinement worth knowing about. A country that has not yet published its figures returns nothing, which is a perfectly successful request, just an empty one. If such a request were treated as finished, that country would never be asked again and its data would be missed when it finally appears. In yearly mode, therefore, a request only counts as finished if it actually returned data. Countries that have not reported yet are asked again on the next run, while countries that already returned data are skipped.

6.6 Step four: download, country by country

The pipeline now loops over the years in range, the countries in the list, and the four trade flows. For each combination it sends one request and saves the result immediately.

Saving immediately, rather than collecting everything in memory and writing at the end, is intentional. It means that an interruption costs at most one request's worth of work.

6.7 Step five: respect the limits of the source

Two limits shape the download, and the pipeline handles both automatically. Because this is the part that most often puzzles newcomers, it has its own section: see section 9.

6.8 Step six: record the progress

After each request, the pipeline writes the outcome to the progress record: whether it succeeded, how many rows arrived, how many requests it actually took, and, in case of failure, the error.

There is one further protection worth understanding. If a run is killed in the very short moment

between saving the data and recording the progress, the data would be saved but not marked as

done, and the next run would download it again, creating duplicates. To prevent this, before retrying any request the pipeline first removes any rows that request may have left behind.

The

effect is that each request's data ends up in the table exactly once, whatever happens.

6.9 Two extra capabilities used by the history mode

Two further functions in `general_extractor.py` belong to the history load rather than the yearly run.

The volume check. Before a large download, this asks the UN how many records each country has

for each year. This serves three purposes: estimating how long the job will take, knowing in advance which countries are large enough to need splitting, and, at the end, comparing the number

of rows expected against the number actually stored.

Update detection. The UN records when each country's figures were last published. This function compares that date with the date on which we downloaded them. If the source has been

republished since our download, that part is marked to be downloaded again, so corrections at the

source are picked up rather than missed. The same function also applies the optional re-download

setting described in section 5.7.

7. The Airflow job: `cp_extraction_comtrade_dag.py`

7.1 The idea

Airflow is the scheduler that runs our pipelines. This file describes the job: which tasks exist,

in which order they run, and what should happen when one of them fails.

The task list is not written out by hand. It is **built automatically from the settings**, which means that adding or removing a classification in the settings file changes the shape of the job without anyone editing this file.

7.2 What the job looks like in yearly mode

In yearly mode the job is simple. There is one task per classification, each downloading the previous year, and a final task that checks the result:

```
download HS → download S1 → download S2 → ... → download EB → final che
```

7.3 What the job looks like in history mode

In history mode the job is larger. The years are divided into batches, for example 2000 to 2009, and there is one task per classification and batch. Two preparatory tasks run first: the volume check and the update detection described in section 6.9.

```
volume check → update detection → download HS 1988-1989 → ... → final che
```

Splitting a multi-day job into batches is what makes it manageable: each batch is a separate task in Airflow, with its own status, its own logs and its own retries, so progress is visible and a problem can be traced to a specific part of the job.

7.4 How the job protects itself

The job is built on the assumption that failures are normal and must not cost work. Five mechanisms work together.

Tasks run one at a time. The download tasks are chained one after another and share a single execution slot. This guarantees that the pipeline never exceeds the source's speed limit by having two tasks download at the same time.

Failed tasks are retried automatically. Airflow retries a failed task several times, waiting longer between each attempt. Because of the progress record, a retry does not repeat completed

work, so retrying is cheap.

One failure does not stop the rest. If a task still fails after its retries, the following tasks run anyway. Losing one classification should not cost the whole job. The failed part is simply picked up on the next run.

A task that reports failure is telling the truth. If any request inside a task could not be completed, the task ends in a failed state rather than quietly finishing. This is what triggers the retries.

A final check makes gaps impossible to miss. The last task reads the progress record and fails the job if anything is still incomplete, listing what is missing. In history mode it also compares the number of rows stored against the number the source said to expect. The job only ends successfully when the data is genuinely complete.

7.5 Scheduling

The job can be triggered manually from the Airflow interface, or given a schedule so that it runs by itself.

If it is scheduled, the timing deserves a moment's thought. Because countries report late, running in January for the year that has just ended would find very little data. A run in the middle of the year catches far more. Running it more than once a year is also perfectly safe: completed work is skipped, and countries that had not yet reported are tried again.

8. The database tables

The pipeline maintains four tables.

Table	What it holds
<code>raw.raw_un_commetrade</code>	All trade data, for every classification and year
<code>public.comtrade_reporters</code>	The official list of reporting countries
<code>public.comtrade_partners</code>	The official list of partner countries

<code>public.comtrade_extraction_checkpoints</code>	The progress record of every request
---	--------------------------------------

8.1 The trade data table

This is the table the transformation layer reads. It holds every classification and every year together in one place, so that downstream work has a single source to point at.

Internally it is divided by classification and then by year, as described in section 6.3. This division is invisible in normal use: queries are written against the table as if it were an ordinary one. For best performance, a query should filter on the classification and the year, because that allows the database to read only the relevant sections.

8.2 The two country lists

These hold the official lists of reporting and partner countries, refreshed from the UN at the start of every run. Besides feeding the download, they are useful reference tables in their own

right, and they carry a marker showing whether each country is still active at the source.

8.3 The progress record

This table has one row per request, recording whether it succeeded, how many rows it returned and

how many requests it actually took.

This table must not be deleted. It is what allows an interrupted run to continue, what prevents

data from being downloaded twice, and what the final check reads to confirm that the data is

complete. Deleting it would not lose any trade data, but the next run would have no way of knowing

what had already been done and would start the entire download again from the beginning.

9. The two limits of the source, and how they are handled

The UN web service imposes two rules. Understanding them explains most of the complexity in the extraction file.

9.1 The speed limit

The service accepts a limited number of requests per second. Exceeding it causes requests to be rejected.

The pipeline paces itself so that it never sends requests faster than allowed. This pacing covers every request it makes, including the extra requests created by the splitting described below. The Airflow job reinforces this by ensuring only one download task runs at a time.

9.2 The size limit, and why it is subtle

A single request returns at most 250,000 rows. The difficulty is that **the service does not warn**

you when it has cut the answer short. A country with a million rows of data returns exactly 250,000 rows, and nothing in the response says that three quarters of the data is missing.

The pipeline therefore uses the row count itself as the signal:

- If a response contains **fewer** than 250,000 rows, everything that exists was returned, and the data is saved.
- If a response contains **exactly** 250,000 rows, it must be assumed to have been cut short.

9.3 How an oversized download is handled

When a response is assumed to have been cut short, the pipeline does not try to salvage it. It

discards the response entirely, because there is no way to know which part of the data was kept

and which was dropped, and saving it would quietly put incomplete data in the warehouse.

Instead, it asks for the same data in two smaller pieces, by splitting the list of partner countries in half and requesting each half separately. If a half is still too large, it is split again, and so on until every piece fits within the limit.

The reason this is safe is worth stating explicitly: every partner country belongs to exactly one

half at every level of splitting. The pieces therefore cover the whole list, with no gaps and no overlaps, and putting them together gives exactly the same data as the single oversized request

would have done if it had not been cut short.

A large trading nation might need four or eight pieces where a small country needs one. The progress record notes when a request had to be split and how many requests it ultimately took, so this is visible after the fact.

10. Running and monitoring the pipeline

10.1 Running it

The job appears in the Airflow interface under the country-profile tag. It can be started manually from there, or left to a schedule if one has been set.

While it runs, each task's log shows which classification and year it is working on, how many requests remain, how many rows have been stored, and a progress percentage. Any request that had to be split is noted in the log as it happens.

10.2 Knowing whether it worked

The simplest indicator is the final task. If the job finishes successfully, the final check has confirmed that every request in the progress record completed. If it ends in failure, the final check lists what is missing.

For more detail, the progress record can be inspected directly. It shows, per classification and year, which requests succeeded, how many rows each returned and what went wrong for any that failed.

10.3 One-time setup on a new environment

If the pipeline is ever moved to a fresh Airflow environment, one piece of setup is required: the execution slot that limits the download tasks to one at a time must exist, or the tasks will not start. It is created in the Airflow administration interface with a single slot. The credentials for the UN web service must also be present in the settings file.

11. Troubleshooting

11.1 The general rule

Run the job again.

Almost every problem is solved this way. Thanks to the progress record, a new run skips everything that already succeeded and retries only what did not. There is no risk of duplicating data and no need to work out where the previous run stopped.

11.2 Common situations

The job failed part of the way through. This is expected from time to time, usually because the source was temporarily unavailable. Run it again.

One classification failed but the others succeeded. This is the job behaving as designed: one failure does not stop the rest. Run it again and only the failed part will be downloaded.

The final check reports missing data. It lists what is missing. Run the job again. If the same part keeps failing across several runs, look at the recorded error for those requests.

A run finished but a country seems to be missing. The most likely explanation is that the country has not published its figures yet, which is normal for recent years. In yearly mode such a country is automatically retried on the next run, so it will be picked up once it reports.

A request could not be split any further. In rare cases a single country and single partner can exceed the size limit on its own. The recorded error identifies exactly which combination is affected. This needs a deliberate decision about how to fetch that particular slice, and should be raised with the data team rather than worked around.

11.3 What not to do

Do not delete the progress record to "start clean". It contains no trade data, but without it the pipeline loses all knowledge of what has been downloaded and would begin the entire history again.

12. Glossary

Classification. A system for grouping products into categories, such as HS for goods or EB for services.

Flow. The direction of trade: import, export, re-import or re-export.

Partner. The country on the other side of a reported transaction.

Period. The time unit of the data. Here, one calendar year.

Progress record. The table holding one row per request, used to resume interrupted runs and to confirm completeness. Also referred to as the checkpoint table.

Reporter. The country that reported the figures.

Reference lists. The official lists of reporting and partner countries published by the UN and copied into our database at the start of every run.

13. Frequently asked questions

Does the year have to be updated every January?

No. The pipeline works out the previous year from the current date by itself.

Why are years as old as 1962 still in the settings if we only download the previous year?

They record when each classification begins. They act as a guard against requesting impossible years, they describe how the table is organised internally, and they would be needed if the history were ever rebuilt. They do not cause old data to be downloaded.

Is it safe to run the job more than once a year?

Yes. Work that has already succeeded is skipped. Running it again is in fact useful, because countries that had not yet published their figures are retried.

Is it safe to run the job twice at the same time?

It should not be necessary, and it is better avoided: two simultaneous runs would compete for the same execution slot and could together exceed the source's speed limit.

What happens if a country is added or renamed at the source?

Nothing needs to be done. The country lists are refreshed from the UN at the start of every run,
so a new country is picked up automatically.

What happens if the UN corrects data for an old year?

In history mode this is detected automatically and the affected part is downloaded again. In yearly mode old years are not re-checked, since the focus is the most recent year. If a correction to an older year needs to be picked up, the optional re-download setting described in section 5.7 can be used.

Can we download only one classification?

Yes. The list of classifications lives in the settings file, and the Airflow job builds its tasks from that list.

How long does the yearly run take?

Far less than the original history load. It covers a single year rather than several decades, so it is a matter of hours rather than days, depending on how much data has been published and how many large countries require splitting.

Where should I start if I want to read the code?

Start with the settings file, which is short and describes the scope of the work. Then read section 6 of this document alongside `run_un_comtrade_api()` in `general_extractor.py`, ignoring the other sources' functions in that file. The Airflow job is best read last, once the extraction logic is familiar.